BIG DATA PROCESSING TOOLS (UNIT 4)

13-Oct-2024

Dr. P. Rambabu, M. Tech., Ph.D., F.I.E.

Topics

Apache Flume is a distributed, reliable, and available service for efficiently collecting, aggregating, and moving large amounts of streaming data into Hadoop. It's designed to handle high-throughput data flows and is particularly useful in environments where data is generated from multiple sources.

Data Sources for Flume

Log Files: Many applications generate log files. Flume can monitor these files and collect data as it gets written.

Syslog: Flume can aggregate logs from various network devices, servers, and applications using the Syslog protocol.

HTTP Sources: Flume can ingest data through HTTP requests, allowing for real-time data ingestion from web applications or other sources that can send HTTP requests.

JDBC Sources: Flume can read data directly from relational databases using JDBC, enabling it to collect data from traditional database systems.

Kafka: Flume can consume messages from Kafka topics, allowing it to integrate with Kafka's messaging system.

NoSQL Data Management

NoSQL databases are designed for storing, retrieving, and managing unstructured or semi-structured data.

Unlike traditional relational databases, NoSQL databases do not rely on a fixed schema, making them more flexible and suitable for handling large volumes of diverse data types.

NoSQL emerged to address the limitations of traditional databases in handling big data, high user loads, and real-time processing.

Types of NoSQL Databases

1. Document-Oriented Databases: Store data as documents (often in JSON or BSON format). Each document can have a different structure, making it ideal for unstructured data. Example: MongoDB, CouchDB.

2. Key-Value Stores: Use a simple key-value pair to store data. These are highly performant for caching and session management. Example: Redis, Amazon DynamoDB

3. Column-Family Stores: Store data in columns rather than rows, enabling efficient data retrieval for large datasets. Example: Apache Hbase, Apache Cassandra, Google Bigtable

4. Graph Databases: Store data in the form of nodes and edges, which represent entities and their relationships, respectively. They are well-suited for relationship-intensive data. Example: Neo4j, Amazon Neptune, Microsoft Azure Cosmos DB

Query Model for Big Data

In NoSQL databases, querying is often more flexible than in traditional SQL databases. Instead of using a rigid query language like SQL, NoSQL databases may provide:

- **Key-based lookups** for retrieving data using unique keys.
- **Document-based queries** for retrieving documents based on certain criteria (as in MongoDB).
- **MapReduce** operations for large-scale data processing.
- **Graph traversals** for exploring relationships in graph databases

Benefits of NoSQL

Scalability: Easily scales horizontally by adding more servers.

Flexibility: Can handle various data formats (structured, semi-structured, unstructured).

High Availability: Designed to operate across distributed systems, ensuring data is available even during node failures.

Cost-Effective: Often open-source and can run on commodity hardware, making it cheaper than traditional database solutions.

MongoDB

MongoDB is a popular document-oriented NoSQL database that stores data in JSON-like documents, making it easy to work with unstructured data. It offers features such as:

- **Schema Flexibility:** Allows different structures for different documents in the same collection.
- **Indexing**: Supports indexing to speed up query performance.
- **Aggregation Framework:** Provides powerful tools for data processing and analysis.
- **Horizontal Scaling:** Enables easy distribution of data across multiple servers.

MongoDB is widely used in web applications, real-time analytics, and other scenarios where flexible schema design and scalability are crucial.

MongoDB provides a connector for Hadoop that enables Hadoop applications to interact with MongoDB data. The MongoDB Hadoop Connector allows you to use Hadoop frameworks like MapReduce, Hive, and Spark to perform data analytics.

HBase Data Model

HBase is a distributed, column-oriented NoSQL database built on top of the Hadoop ecosystem. It is designed for handling large amounts of sparse data across many commodity servers. Here's an overview of its data model:

Table Structure:

- HBase stores data in tables, where each table consists of rows and columns.
- Each table has a unique name and is identified by a row key.

Row Key:

- The row key is a unique identifier for each row in a table.
- It is stored in sorted order, enabling quick lookups and scans.

HBase Data Model

Column Families:

- Each table can have one or more column families, which are groups of related columns.
- Column families must be defined when creating a table, and all columns within a family are stored together on disk.

Columns:

- HBase supports a flexible schema, meaning that columns can be added dynamically.
- Columns within a column family can have different data types and structures.

Versions:

- HBase supports multiple versions of data for each cell, allowing you to keep historical data.
- You can configure how many versions to retain for each column family.

Implementations and Hbase Clients

Implementations:

- **Real-time Analytics:** Ideal for scenarios requiring real-time read and write access to large datasets.
- **Time-Series Data:** Useful for storing and analyzing time-series data due to its ability to handle large amounts of data with time-based keys.
- **Big Data Processing:** Often used alongside Hadoop for processing massive datasets.

HBase Clients:

HBase provides several client interfaces for interacting with its tables:

Java API: The primary client API for HBase, allowing developers to perform CRUD (Create, Read, Update, Delete) operations programmatically in Java applications.

REST API: Provides a way to interact with HBase through HTTP requests. It is useful for applications that require language-agnostic access to HBase data.

Avro API: Allows for serialization of data in a compact binary format. It can be used to communicate with HBase across different platforms.

Praxis

Praxis refers to the application of theoretical knowledge in practical situations. In the context of HBase, praxis could involve:

- Implementing HBase in real-world projects, such as data warehousing or real-time analytics systems.
- Understanding best practices for HBase table design, including the choice of row keys and column families to optimize performance.
- Experimenting with various HBase clients and APIs to determine the most effective way to interact with HBase in different application scenarios.

By leveraging HBase's capabilities, organizations can effectively handle large-scale data storage and processing requirements.

Sample HBase Table Structure

Let's create a table called `Books` with the following characteristics:

- **Table Name**: `Books`
- **Column Family**: `details`
    - **Columns**:
        - `title`
        - `author`
        - `published_year`
        - `genre`

Sample Data

Here's a representation of some sample data that can be inserted into the `Books` table:

Row Key	Column Family	Column	Value
book1	details	title	To Kill a Mockingbird
book1	details	author	Harper Lee
book1	details	published_year	1960
book1	details	genre	Fiction
book2	details	title	1984
book2	details	author	George Orwell
book2	details	published_year	1949
book2	details	genre	Dystopian

Methods to Load Data into HBase

1. HBase Shell:

Direct Insertion: Use commands like put to manually insert data into HBase. Suitable for small datasets or quick tests.

2. Apache Hadoop MapReduce:

MapReduce Jobs: Write MapReduce jobs to read data from various sources (e.g., text files) and write it to HBase. Ideal for large datasets with complex processing.

3. Apache Spark:

Spark with HBase Connector: Use Spark to read data from multiple sources and write it to HBase. Great for real-time analytics and big data processing.

4. Apache Sqoop:

Import from RDBMS: Use Sqoop to import data from relational databases directly into HBase, efficiently handling connections and loading.

Methods for Data Analytics on HBase

1. HBase Shell:

Direct Queries: Use basic commands to perform quick checks and small-scale analytics directly from the HBase shell.

2. Apache Spark:

Spark with HBase Connector: Leverage Spark's processing capabilities to read data from HBase, enabling advanced analytics through aggregations and transformations.

3. Apache Hive:

Hive on HBase: Create external tables in Hive that point to HBase, allowing SQL-like queries for batch processing and analysis.

4. Apache Pig:

Pig Scripts: Use Apache Pig for writing scripts to read from HBase and perform analytics without needing to handle MapReduce complexities.

5. BI Tools Integration:

Business Intelligence Tools: Connect BI tools (e.g., Tableau, Qlik) to HBase directly or through Hive for data visualization and analysis.

HBase Table: `books`

Row Key	Column Family	Qualifier	Value
book:1	info	title	To Kill a Mockingbird
book:1	info	author	Harper Lee
book:1	info	genre	Fiction
book:2	info	title	1984
book:2	info	author	George Orwell
book:2	info	genre	Dystopian

Here are two records for a MySQL table of books:

ID	Title	Author	Genre
1	"To Kill a Mockingbird"	"Harper Lee"	"Fiction"
2	"1984"	"George Orwell"	"Dystopian"

MongoDB - Sample Books data in JSON format:

```
[
    {
        "ID": 1,
        "Title": "To Kill a Mockingbird",
        "Author": "Harper Lee",
        "Genre": "Fiction"
    },
    {
        "ID": 2,
        "Title": "1984",
        "Author": "George Orwell",
        "Genre": "Dystopian"
    }
]
```

Apache Hive

Apache Hive is a data warehouse system built on top of Hadoop that facilitates reading, writing, and managing large datasets residing in distributed storage using SQL-like syntax.

It provides an abstraction layer to query and manage data using HiveQL (Hive Query Language), which is similar to SQL. Hive is particularly suited for batch processing and is used to query structured data stored in Hadoop's HDFS or other storage systems like Amazon S3.

Feature	Traditional Databases	Apache Hive
Data Storage	Relational databases store data in structured tables.	Stores data in HDFS (Hadoop Distributed File System) or other distributed file systems.
Schema	Schema-on-write (defined before data is written).	Schema-on-read (schema is applied at the time of data retrieval).
Data Volume	Typically handles GBs to TBs of data.	Designed for petabyte-scale data processing.
Query Language	Uses SQL (Structured Query Language).	Uses HiveQL, a SQL-like query language.
Latency	Low-latency, suitable for OLTP (Online Transaction Processing).	High-latency, designed for OLAP (Online Analytical Processing) tasks.
Concurrency	Supports high concurrency for many simultaneous users.	Not optimized for high concurrency; focuses on batch processing.

HiveQL (Hive Query Language)

HiveQL is the SQL-like query language used in Hive. It allows you to query data in Hadoop using familiar SQL-like syntax. HiveQL supports most standard SQL operations like SELECT, JOIN, GROUP BY, and ORDER BY. Hive translates these queries into MapReduce jobs, Tez jobs, or Spark jobs for execution.

Key HiveQL Features:

1. Basic SQL Syntax: HiveQL follows standard SQL, making it easy for those familiar with SQL to use.

2. Partitioning: Data in Hive can be partitioned based on certain columns, which helps optimize query performance by scanning only relevant partitions.

3. Bucketing: Hive supports bucketing, where data is divided into fixed buckets based on the value of a column, which helps with joins and sampling.

4. Joins: Hive supports SQL-like INNER, LEFT OUTER, RIGHT OUTER, and FULL OUTER joins

5. Windowing and Analytics: HiveQL supports window functions like ROW_NUMBER(), RANK(), and LEAD() to perform analytical queries.

Feature	Partitioning	Bucketing
Purpose	Divides data into sub-directories by column	Divides data into fixed buckets using a hash
Best for	Low-cardinality columns (e.g., country)	High-cardinality columns (e.g., user_id)
Benefit	Limits data scanning for specific partitions	Enhances join performance on bucketed columns
Downside	Too many partitions if high cardinality	Predefined bucket count, less dynamic
Example	Partitions by `country`	Buckets by `user_id` into N buckets

In Hive, **partitioning** allows you to divide a large table into smaller, more manageable pieces based on the values of specific columns. This approach helps improve query performance by allowing Hive to scan only the relevant partitions instead of the entire table.

In the example you provided, the table employees is partitioned by the year column. Here's a breakdown of how it works:

Table Structure:

```
CREATE TABLE employees (
    id INT,
    name STRING,
    age INT,
    department STRING
)
PARTITIONED BY (year INT);
```

Purpose of Partitioning:
Partitioning helps optimize queries. For example, if you're frequently querying employee data for specific years, Hive will only scan the partition(s) related to those years instead of the entire table.

Example Scenario:
Let's say you have employee data from 2021, 2022, and 2023. In this case, the table will have three partitions: one for each year (2021, 2022, 2023). If you run a query to fetch data for employees in 2022, Hive will scan only the 2022 partition, reducing the amount of data read and improving performance.

Query Example:
If you want to query for employees in the year 2022:

```
SELECT * FROM employees WHERE year = 2022;
```

Hive will only scan the partition corresponding to the year 2022, making the query faster than scanning the entire dataset.

In Hive, **bucketing** is a technique used to divide data into a fixed number of buckets based on the value of one or more columns. It is especially useful in optimizing queries involving joins and sampling.

How Bucketing Works:

When a table is bucketed, rows with the same bucketed column value are stored in the same bucket (a separate file in the partition). Hive uses a hash function to determine which bucket each row should go into. This is different from partitioning, where data is split based on column values into partitions.

Example Syntax for Bucketing:

```
CREATE TABLE employees (
    id INT,
    name STRING,
    age INT,
    department STRING
)
CLUSTERED BY (id) INTO 4 BUCKETS;
```

Key Concepts:

CLUSTERED BY: Specifies the column to use for bucketing. In this case, the id column is used.

INTO N BUCKETS: Specifies the number of buckets (here 4) into which the data will be divided.

Benefits of Bucketing:

- Optimized Joins: When both tables in a join are bucketed on the join column, Hive can efficiently join the data by reading only corresponding buckets, reducing data shuffling.
- Sampling: Bucketing allows sampling of data by reading a few buckets instead of the entire dataset, which is useful for data analysis and testing.

Example Query:

For instance, if you want to join two tables, and both are bucketed by id into the same number of buckets:

```
SELECT e1.name, e2.department
FROM employees e1
JOIN employee_details e2
ON e1.id = e2.id;
```

This join will be more efficient as Hive will match rows from corresponding buckets, avoiding

In HiveQL, **window functions** (also called **analytic functions**) provide a way to perform calculations across a set of rows that are related to the current row, while retaining the ability to return multiple rows. These functions allow you to perform more advanced analytical queries, such as rankings, running totals, and lead/lag analysis.

Common Hive Window Functions:

ROW_NUMBER(): Assigns a unique sequential integer to rows within a partition of a result set, starting at 1 for the first row.

RANK(): Assigns a rank to each row within a partition of the result set. Rows with equal values receive the same rank, but the next rank will be incremented based on the number of identical rankings.

DENSE_RANK(): Similar to RANK(), but without gaps in the ranking sequence when there are ties.

LEAD(): Accesses data from the next row in the result set without the need for a self-join.

LAG(): Similar to LEAD(), but accesses data from the previous row.

Syntax:

```
SELECT id, name, department, salary,
  ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC) AS row_num,
  RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS rank,
  LEAD(salary, 1) OVER (PARTITION BY department ORDER BY salary DESC) AS next_salary
FROM
  employees;
```

Example Use Case:

If you want to rank employees within each department based on their salary:

```
SELECT id, name, department, salary,
  RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS rank
FROM employees;
```

This query will give you the rank of each employee within their department, ordered by

In Hive, working with tables, performing queries, and creating User Defined Functions (UDFs) are fundamental aspects of managing and analyzing large datasets. Here's an overview of each concept:

1. Tables in Hive:

Hive tables are analogous to tables in a relational database but are stored in HDFS or other file systems like Amazon S3.

Types of Hive Tables:

Managed (Internal) Tables: Hive manages the data, and the data is deleted when the table is dropped.

External Tables: The data resides outside of Hive's control (e.g., in HDFS, S3). When the table is dropped, the data remains intact.

Partitioned Tables: Data is partitioned based on a column, optimizing query performance by scanning only relevant partitions.

Bucketed Tables: Data is divided into fixed buckets based on a hash function applied to a column. This is useful for joins and sampling.

Querying Data in Hive:

Hive uses a query language called HiveQL, which is similar to SQL. You can perform basic operations like SELECT, INSERT, UPDATE, and DELETE, as well as more complex operations like joins, aggregations, and subqueries.

Basic Query:

```
SELECT id, name, department
FROM employees
WHERE department = 'HR’;
```

Join Query:

```
SELECT e.id, e.name, d.department_name
FROM employees e
JOIN departments d
ON e.department = d.department_id;
```

Aggregation Query:

```
SELECT department, COUNT(*)
FROM employees
GROUP BY department;
```

Complex Query with Partition and Bucketing:

```
SELECT id, name, salary
FROM employees
WHERE year = 2023
AND bucket_number = 2;
```

In this case, the query will scan only the relevant partition (year = 2023) and bucket (bucket_number = 2), improving performance.

User Defined Functions (UDFs):

Hive provides a rich set of built-in functions for operations like string manipulation, arithmetic, and date processing. However, for custom operations, Hive allows you to define User Defined Functions (UDFs) in Java.

How UDFs Work:

A UDF in Hive is essentially a Java function that you can call in your Hive queries. UDFs are useful when the built-in functions do not meet your requirements. They allow for custom logic and extend the capabilities of Hive.

Steps to Create a UDF:

Write the UDF in Java: Create a class that extends org.apache.hadoop.hive.ql.exec.UDF and implements the evaluate() method.
Compile the UDF: Compile your UDF and package it as a JAR file.
Add the JAR to Hive: Load the JAR into Hive using the ADD JAR command.
Create a Temporary Function: Register the UDF in Hive using the CREATE TEMPORARY FUNCTION statement.

Example of a Simple UDF (in Java):

```
public class UppercaseUDF extends UDF {
    public Text evaluate(Text input) {
        if (input == null) {
            return null;
        }
        return new Text(input.toString().toUpperCase());
    }
}
```

Steps to Use the UDF in Hive:

```
ADD JAR /path/to/udf.jar;
CREATE TEMPORARY FUNCTION to upper AS 'com.example.hive.udf.UppercaseUDF';
SELECT to upper(name) FROM employees;
```

Assignment:

1. Give list of Built-in Functions in Hive.
2. What is the purpose of a User Defined Function (UDF) in Hive, and when might it be necessary to create one instead of using built-in functions?
3. Develop a UDF that calculates the Age of Employee based on their Date of Birth. How would you register and use this UDF on a sample table?

Apache Derby is often used in Apache Hive as the default metastore database when setting up. The **metastore** is crucial in Hive because it stores metadata about the databases, tables, partitions, schemas, and other components. Hive needs to understand the structure and layout of data stored in Hadoop.

Here's why Derby is used in Hive, especially in local or small-scale setups:

1. Embedded Database for Easy Setup
Derby can run in embedded mode, meaning it doesn't require a separate database server process. This is convenient for testing, development, or single-user environments.

2. Lightweight and Java-Based
Since Hive is also Java-based, Derby's Java compatibility makes it easy to integrate. Derby is a lightweight option that doesn't consume as many resources, which suits local and lightweight setups.

3. Single-User Mode Support for Local Development
Derby's embedded mode works well for single-user access, which is adequate for local testing and development.

4. Quick Prototyping
For quick prototyping or for developers who want to experiment with Hive, Derby provides an easy-to-deploy metastore option without needing external database dependencies.
This approach allows users to set up Hive with minimal steps, which is ideal for learning and small-scale testing.

5. Switching to Production-Ready Databases
Use Derby: Only for development, testing, and single-user scenarios to keep things simple.
Switch to MySQL or PostgreSQL: For production or any environment where Hive needs to support multiple users or larger workloads.

Sample Metadata stored in Derby Database for Hive

1. `DATABASES` Table

DB_ID	NAME	DB_LOCATION_URI	DESCRIPTION
1	sales_db	hdfs://localhost:9000/user/hive/warehouse/sales_db	Database for sales data
2	analytics_db	hdfs://localhost:9000/user/hive/warehouse/analytics_db	Database for analytics data

2. `TBLS` Table

TBL_ID	DB_ID	TBL_NAME	TBL_TYPE	OWNER	CREATE_TIME
1	1	sales_records	MANAGED_TABLE	user1	1672531200
2	2	customer_data	EXTERNAL_TABLE	user2	1672617600

3. `COLUMNS_V2` Table

CD_ID	COLUMN_NAME	TYPE_NAME	COMMENT
1	sales_id	int	Primary key for sales
1	date	string	Date of the sale
2	customer_id	int	Unique customer ID
2	customer_name	string	Name of the customer

Sqoop

Apache Sqoop (SQL-to-Hadoop) is a powerful tool in the Hadoop ecosystem designed for transferring large volumes of data between relational databases (e.g., MySQL, PostgreSQL, Oracle) and Hadoop. It serves as a bridge, allowing organizations to combine the structured data stored in traditional databases with the unstructured data-processing capabilities of Hadoop.

Purpose of Sqoop in the Hadoop Ecosystem

Sqoop's primary purpose is to facilitate the import and export of data between Hadoop and relational databases. It provides an efficient way to transfer data with minimal overhead, eliminating the need to write complex custom scripts for data migration. Sqoop automates the process, making it highly suitable for ETL (Extract, Transform, Load) workflows.

How Sqoop Works

Sqoop uses connectors and the JDBC (Java Database Connectivity) protocol to connect with databases. It translates SQL queries into MapReduce jobs, which are then executed to import/export data between the database and the Hadoop Distributed File System (HDFS).

Key Functions of Sqoop

Data Import: Sqoop can import entire tables or specific columns from a database into HDFS, Hive, or HBase.

Data Export: It exports processed or transformed data from HDFS back to relational databases.

Incremental Data Loading: Sqoop supports incremental imports, allowing only new or updated records to be transferred, which is beneficial for keeping datasets current.

Parallel Processing: Sqoop splits data into multiple parallel tasks, ensuring that data transfers are fast and scalable.

Sqoop's Integration with Other Hadoop Ecosystem Components

Sqoop enhances the utility of other components in the Hadoop ecosystem by serving as an entry point for structured data. Here's how it works with other Hadoop tools:

1. HDFS (Hadoop Distributed File System): The primary storage layer in Hadoop where Sqoop stores imported data, enabling distributed processing and storage across Hadoop clusters.
2. Hive: Data imported from Sqoop can be directly stored in Hive tables. This allows analysts and data scientists to query relational data with HiveQL, making data exploration and reporting accessible through SQL-like syntax.
3. HBase: Sqoop can import data into HBase tables, enabling low-latency, real-time read and write access to the imported data.
4. MapReduce: Sqoop's own data import/export operations are based on MapReduce, utilizing Hadoop's distributed processing capabilities to handle large-scale data transfers efficiently.
5. Apache Spark: Data imported by Sqoop into HDFS can be further processed by Spark, which can handle data transformations, machine learning, and advanced analytics on the imported datasets.

Sqoop Workflow in a Data Pipeline

Sqoop is often part of larger ETL workflows in a Hadoop environment. Here's a typical pipeline where Sqoop plays a crucial role:

Extract:

Sqoop connects to a relational database and extracts data based on user specifications. The extracted data is loaded into HDFS, Hive, or HBase for further processing.

Transform:

Data transformations can be performed using HiveQL, Pig, or Spark, allowing the data to be cleaned, transformed, and prepared for analysis.

Load:

After transformations, the processed data can be exported back to the relational database using Sqoop's export functionality or used in Hadoop's storage layers for analysis.

Example of Using Sqoop

Here's an example of a common Sqoop import command that imports data from a MySQL database into a Hive table:

```
sqoop import \
--connect jdbc:mysql://hostname/dbname \
--username dbuser \
--password dbpass \
--table employees \
--target-dir /user/hadoop/employees \
--hive-import \
--create-hive-table \
--hive-table default.employees_hive
```

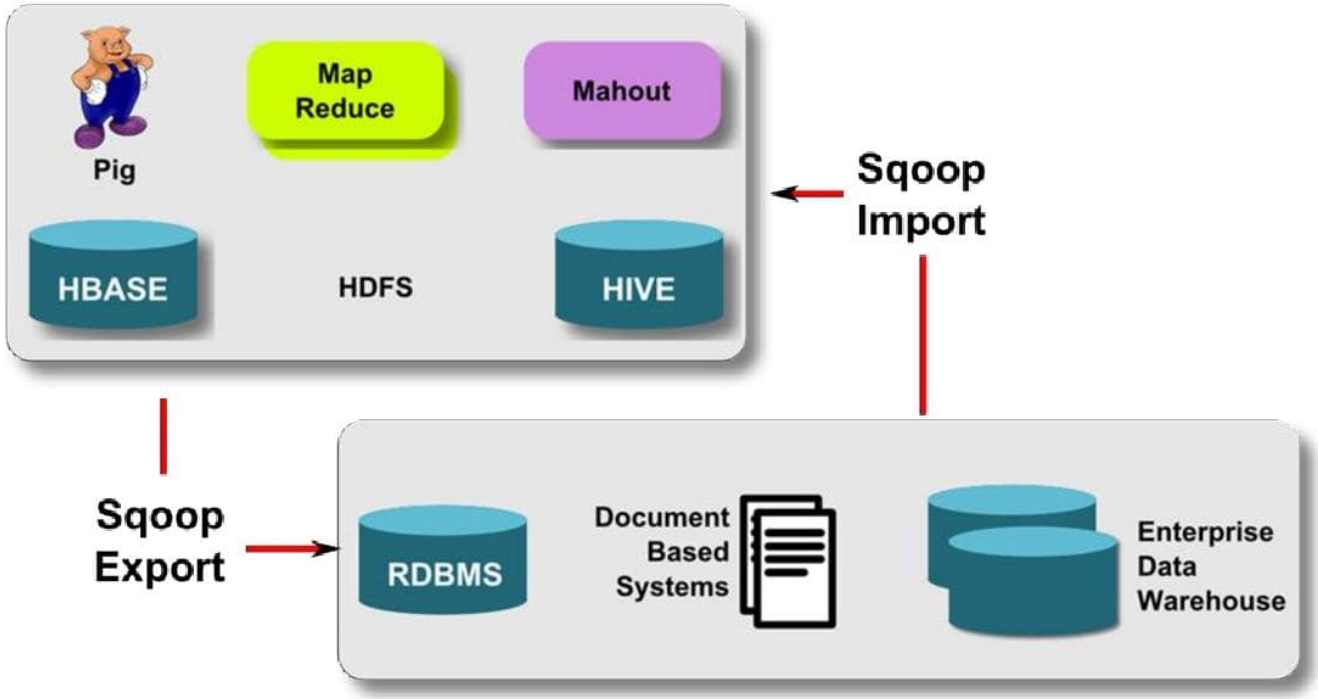
Flume

Apache Flume is a distributed, reliable, and available service for efficiently collecting, aggregating, and moving large amounts of streaming data into Hadoop. It's designed to handle high-throughput data flows and is particularly useful in environments where data is generated from multiple sources.

Use Cases for Apache Flume

1. Log Aggregation: Collecting logs from various servers and applications into a centralized Hadoop system for analysis.
2. Streaming Data Ingestion: Ingesting data from social media feeds, IoT devices, or web applications in real time.
3. Data Migration: Moving data from legacy systems to modern big data platforms like Hadoop or cloud storage.

Apache Sqoop



Components of Flume Architecture

Flume's architecture consists of several key components:

Source: This component is responsible for receiving data from various sources. A source pulls data from an external system and converts it into Flume events. Common source types include Spool Directory Source, Http Source, and JDBC Source.

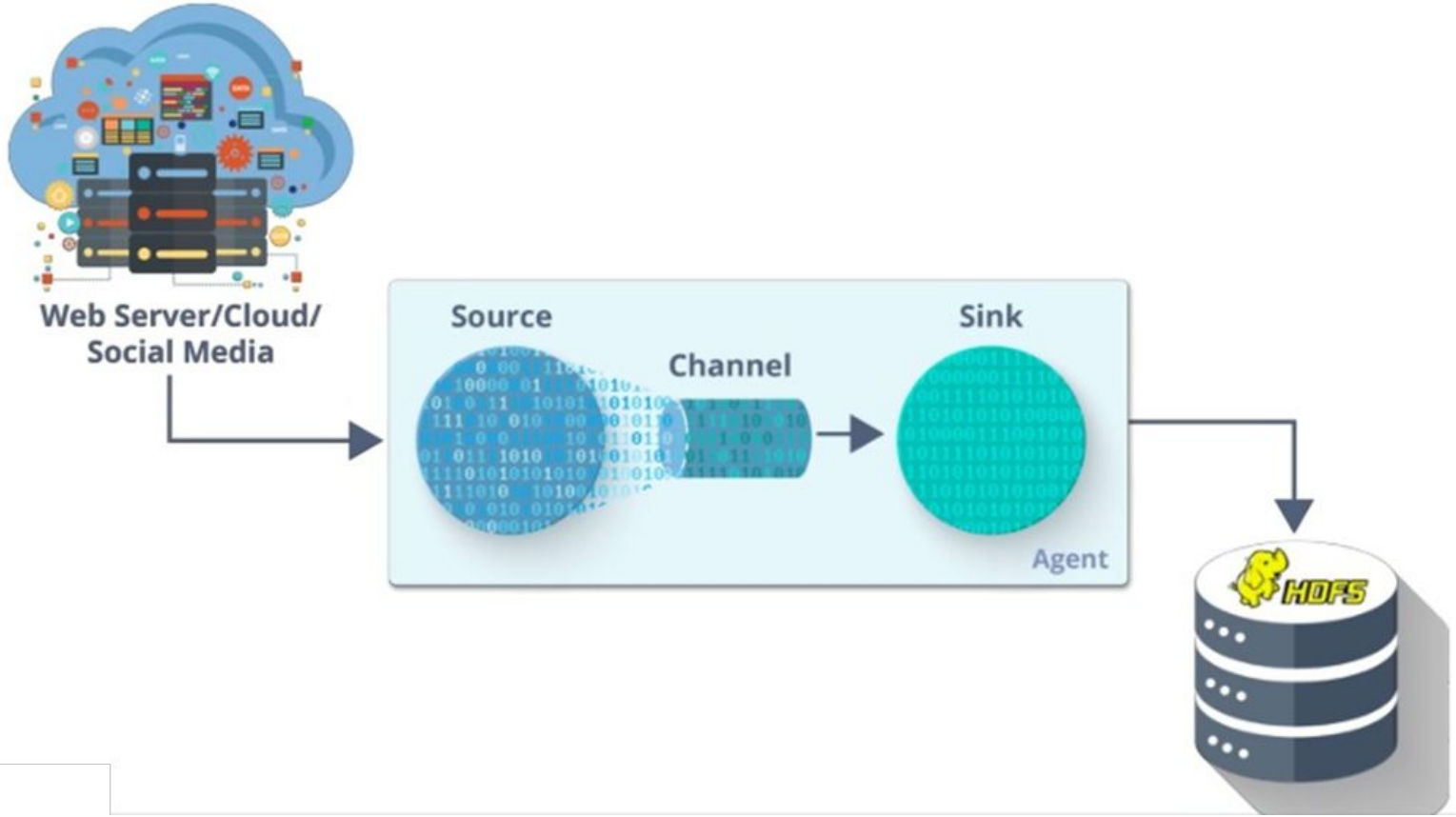
Channel: A channel acts as a buffer between the source and sink. It temporarily stores the events until they are processed by the sink. Channels can be either memory-based (faster but less durable) or file-based (slower but more durable).

Sink: The sink component consumes the events from the channel and delivers them to a final destination (e.g., HDFS, HBase, or another data store). Sinks can also be configured to perform batch processing.

Agent: An agent is a Java process that hosts the sources, channels, and sinks. An agent can collect data from various sources and process it through its channels and sinks.

Configuration: Flume is configured using a simple text file that specifies the sources, channels, and sinks. This configuration allows you to define how data flows through your Flume application.

Apache Flume Architecture



Flume Data Flow

The data flow in Flume is generally as follows:

1. Data is generated at a source (e.g., application logs, user activity).
2. Flume agent reads this data using its configured source.
3. The source sends the data to a channel for temporary storage.
4. A sink reads from the channel and sends the data to a destination, such as HDFS or a database.

Dr. Rambabu Palaka
Professor
School of Engineering
Malla Reddy University, Hyderabad

Mobile: +91-9652665840
Email: drrambabu@mallareddyuniversity.ac.in